

XI. Flexbox — mise en page 1D

FLEXBOX EN UNE PHRASE

Un système de mise en page CSS pour disposer des éléments **SUR UN SEUL AXE** (ligne ou colonne), remplace définitivement les hacks pré-2015 (`float` , `inline-block`).

Sommaire

- **Introduction**
 - 1. Objectifs du chapitre
 - 2. En pratique — situation professionnelle
 - 3. Quand l'utiliser, quand l'éviter
- **A. Pourquoi Flexbox ?**
- **B. Les bases : display: flex**
 - 1. Activer Flexbox
 - 2. Les 2 axes
 - 3. flex-direction
 - 4. flex-wrap
- **C. Alignement et répartition**
 - 1. justify-content (axe principal)
 - 2. align-items (axe perpendiculaire)
 - 3. gap — l'espacement propre
 - 4. Centrer quelque chose, enfin facilement
- **D. Propriétés des enfants**
 - 1. flex-grow, flex-shrink, flex-basis
 - 2. Exemple concret : navbar
 - 3. align-self
- **Conclusion**
 - 1. À retenir
 - 2. Pour aller plus loin

Introduction

1. Objectifs du chapitre

À l'issue de ce chapitre, vous serez capable de :

- **Activer** Flexbox avec `display: flex` et comprendre ses 2 axes.
- **Appliquer** `justify-content`, `align-items`, `gap` pour l'alignement et la répartition.
- **Centrer** n'importe quel élément en 3 lignes de CSS.
- **Utiliser** `flex-grow`, `flex-shrink`, `flex-basis` sur les enfants.
- **Choisir** Flexbox pour une mise en page 1D (vs Grid pour 2D — voir chapitre XII).

2. En pratique — situation professionnelle

Le designer vous livre une navbar (logo à gauche, menu à droite) et une barre d'action (boutons alignés). Flexbox règle ces deux cas en 5 lignes de CSS chacun. Sans Flexbox, les techniques pré-2015 (`float`, `clear`) demandaient des heures de hacks.

3. Quand l'utiliser, quand l'éviter

✓ Utiliser pour	✗ Éviter quand
Navbar, barre d'actions, mise en page 1D	Grille de cards 2D → préférer CSS Grid (chap. XII)
Centrage vertical (<code>justify-content</code> + <code>align-items</code>)	Layout typographique flow (texte normal)
Distribution équitable (<code>flex: 1</code>) entre items	Compatibilité IE10/11 (rare en 2026)

A. Pourquoi Flexbox ?

Avant 2015, pour aligner 3 éléments horizontalement, il fallait :

```
.item {  
  float: left;  
  width: 33.33%;  
}  
.clearfix::after {  
  content: "";  
  display: table;  
  clear: both;  
}
```

Ou utiliser des **tableaux** pour la mise en page — anti-pattern connu.

Avec Flexbox, une seule ligne suffit :

```
.container { display: flex; }
```

Flexbox est idéal pour une mise en page **1D** (alignement sur **un** axe, ligne OU colonne). Pour 2D (grilles avec lignes ET colonnes), voir Grid au chapitre XII.

B. Les bases : `display: flex`

1. Activer Flexbox

```
.container {  
  display: flex;  
}
```

Dès lors, **tous les enfants directs** du conteneur deviennent des **flex items** placés côte à côte.

2. Les 2 axes



Avec `flex-direction: column`, les axes sont inversés (main vertical, cross horizontal).

- **Main axis** (axe principal) — sens du flux, contrôlé par `flex-direction`.
- **Cross axis** (axe perpendiculaire) — orthogonal au main axis.

3. flex-direction

```
.container {  
  display: flex;  
  flex-direction: row;          /* défaut : horizontal, left → right */  
  /* flex-direction: column;    /* vertical, top → bottom */  
  /* flex-direction: row-reverse; /* horizontal, right → left */  
  /* flex-direction: column-reverse; */  
}
```

4. flex-wrap

Par défaut, les items ne **vont pas à la ligne** même si ça déborde. Pour permettre le retour :

```
.container {  
  display: flex;  
  flex-wrap: wrap;  
}
```

C. Alignement et répartition

1. justify-content (axe principal)

Distribue l'espace **le long du main axis**.

```
.container { justify-content: flex-start; } /* défaut */
.container { justify-content: center; } /* centrés */
.container { justify-content: flex-end; } /* collés à la fin */
.container { justify-content: space-between; } /* rien aux extrémités */
.container { justify-content: space-around; } /* demi-espaces aux extrémités */
.container { justify-content: space-evenly; } /* uniformément */
```

2. align-items (axe perpendiculaire)

```
.container { align-items: stretch; } /* défaut : étire */
.container { align-items: flex-start; } /* alignés au début */
.container { align-items: center; } /* centrés */
.container { align-items: flex-end; } /* à la fin */
.container { align-items: baseline; } /* ligne de base du texte */
```

3. gap — l'espacement propre

```
.container {
  display: flex;
  gap: 1rem;
}
```

♦ `gap` est le secret d'un CSS moderne propre

Avant : `margin-right: 1rem` sur tous les items + retirer le dernier avec `:last-child`. Avec `gap` : **une seule ligne**. Supporté partout depuis 2021.

4. Centrer quelque chose, enfin facilement

Le Saint Graal de CSS :

```
.parent {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100vh;  
}
```

→ L'enfant est **parfaitement centré** vertical + horizontal.

✗ Sans largeur

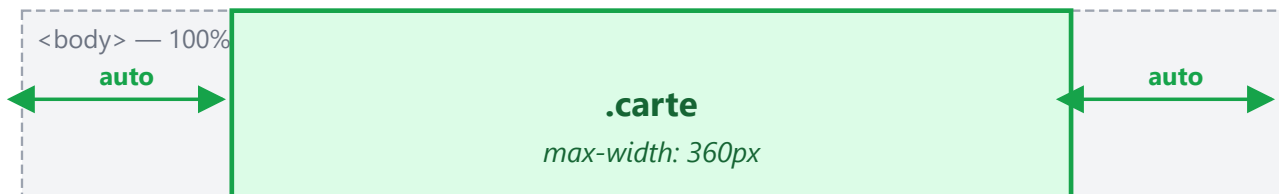
```
.carte { margin: 0 auto; }
```

.carte

déjà à 100% → margin auto n'a rien à répartir

✓ Avec max-width

```
.carte { max-width: 360px; margin: 0 auto; }
```



D. Propriétés des enfants

1. flex-grow, flex-shrink, flex-basis

```
.item {  
  flex-grow: 1;      /* prend l'espace disponible */  
  flex-shrink: 1;    /* se réduit si nécessaire */  
  flex-basis: auto;   /* taille de base */  
  
  /* Raccourci : flex: grow shrink basis */  
  flex: 1 1 auto;  
  flex: 1;            /* = flex: 1 1 0 (souvent suffisant) */  
}
```

2. Exemple concret : navbar

```
<nav class="navbar">  
  <div class="logo">Logo</div>  
  <ul class="menu">  
    <li>Accueil</li>  
    <li>Blog</li>  
    <li>Contact</li>  
  </ul>  
</nav>
```

```
.navbar {  
  display: flex;  
  align-items: center;  
  padding: 1rem 2rem;  
}  
  
.logo { flex: 0 0 auto; }  
  
.menu {  
  display: flex;  
  gap: 2rem;  
  margin-left: auto; /* pousse le menu à droite */  
  list-style: none;  
}
```


♦ `margin-left: auto` sur un flex item

Dans un conteneur flex, une `margin: auto` d'un côté **pousse** l'item vers le côté opposé. Classique pour séparer un logo et un menu dans une navbar.

3. `align-self`

Permet à **un seul item** d'avoir un alignement différent des autres :

```
.item-special {  
  align-self: flex-end;  
}
```

Conclusion

1. À retenir

- `display: flex` + `justify-content` + `align-items` + `gap` = 90 % des cas d'alignement résolus.
- **Centrer** : `display: flex; justify-content: center; align-items: center;` sur le parent.
- `gap` remplace les `margin` bricolés entre items.
- `flex: 1` sur un item pour qu'il prenne l'espace disponible.
- `margin: auto` sur un flex item pour le pousser vers un côté (navbar logo/menu).

2. Pour aller plus loin

- **MDN — Flexbox (guide complet)**
- **CSS Tricks — A Complete Guide to Flexbox**
- **Flexbox Froggy** — jeu d'apprentissage en français (30 min, **à faire**).